

Unit 01

Introduction to Database Management Systems

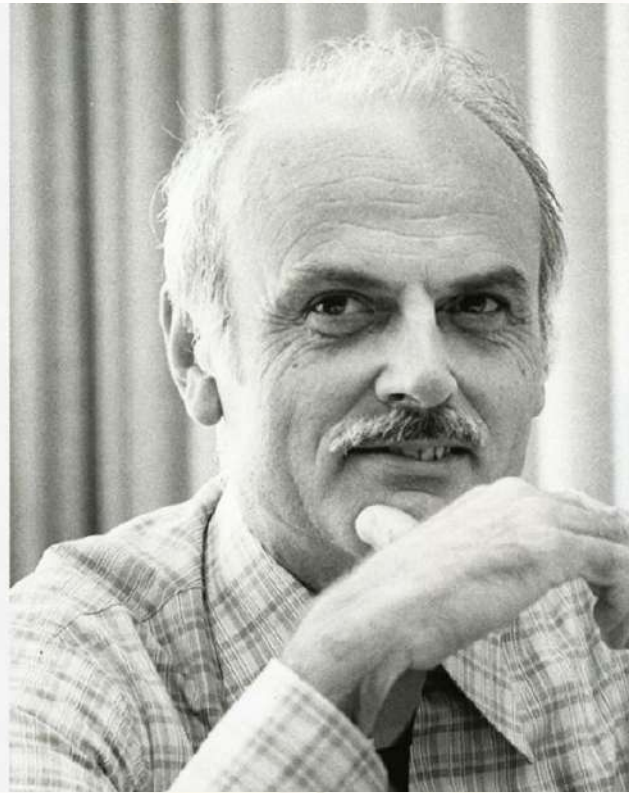
Prof. Ashwini Kumar Mathur
[School of Computer Science and Engineering]

Outlines

1. Database vs File System
2. Database system concept and Architecture
3. Data Model Schema and Instances
4. Data Independence, Database Language and Interfaces: DDL,DML,DCL and TCL
5. Overall Database Structure -
 - Storage Components,
 - Metadata,
 - Users Database Backups and Recovery Mechanisms.

Remembering Edgar F. Codd

The Father of Relational
Database Management
Systems



Research Paper Link: <https://www.seas.upenn.edu/~zives/03f/cis550/codd.pdf>

Notes: [EF Codd key research notes](#)

File System – Overview

- A **File System** stores data in separate files
- Data is managed by the **operating system**
- Each application defines its **own data format**
- No centralized control over data

Examples: Text files, Excel sheets, CSV files

Database System – Overview

- A **Database System** stores data in a structured and organized way
- Managed by a **Database Management System (DBMS)**
- Data is stored in **tables with relationships**
- Centralized control over data

Examples: MySQL, Oracle, PostgreSQL

Key Differences (Conceptual)

Aspect	File System	Database System
Data Storage	Files	Tables
Data Organization	Unstructured / Semi-structured	Structured
Data Control	Application dependent	DBMS controlled
Data Access	Manual	Query-based (SQL)

Data Redundancy

File System

- Same data stored in multiple files
- Leads to **duplication**
- Wastes storage space

Database System

- Redundancy is **controlled**
- Data stored at one place
- Efficient use of storage

Data Consistency

File System

- High chance of inconsistency
- Updating data in one file may not update others

Database System

- Ensures **data consistency**
- Changes are reflected across all applications

Data Security

File System

- Limited security
- File-level access only
- No user-based privileges

Database System

- Strong security mechanisms
- User authentication and authorization
- Role-based access control

Data Sharing

File System

- Difficult to share data
- Multiple users may cause conflicts

Database System

- Easy data sharing
- Supports **multi-user access**
- Concurrency control mechanisms

Data Integrity

File System

- No enforcement of constraints
- Data may become invalid

Database System

- Integrity constraints enforced
- Ensures accurate and valid data

Backup and Recovery

File System

- Manual backup required
- No automatic recovery

Database System

- Automatic backup options
- Recovery from system failures supported

Query Processing

File System

- No query language
- Data retrieval is time-consuming

Database System

- Uses SQL for data retrieval
- Fast and efficient querying

Scalability

File System

- Not suitable for large data
- Performance degrades as data grows

Database System

- Highly scalable
- Handles large volumes of data efficiently

Feature	File System	Database System
Redundancy	High	Low
Security	Low	High
Consistency	Poor	Good
Scalability	Limited	High
Query Support	No	Yes

File Systems

A **File System** is a tool which facilitates **storing, accessing and modifying data from numerous files** in a computer system.



All data is stored in the form of files. All files are categorized and sorted accordingly. The file names are closely related to one another and are organised in such a way that they are easily accessible.

Files like .txt, .jpg, and .docx, as well as **structured data types like .xml and .json,** may be used by a file system. For organizing or more complex administration needs, files can be placed in different system directories.

Examples of file systems include storage media like hard disks, pen drives, etc, that may contain data such as music files, videos, documents and e.t.c.

DBMS VS File Systems

FILE SYSTEM	DBMS
Used to manage and organise the files stored in the hard disk of the computer	A software to store and retrieve the user's data
Redundant data is present	No presence of redundant data
Query processing is not so efficient	Query processing is efficient
Data consistency is low	Due to the process of normalisation, the data consistency is high
Less complex, does not support complicated transactions	More complexity in managing the data, easier to implement complicated transactions
Less security	Supports more security mechanisms
Less expensive in comparison to DBMS	Higher cost than the File system
Does not support crash recovery	Crash recovery mechanism is highly supported

Basic Definitions

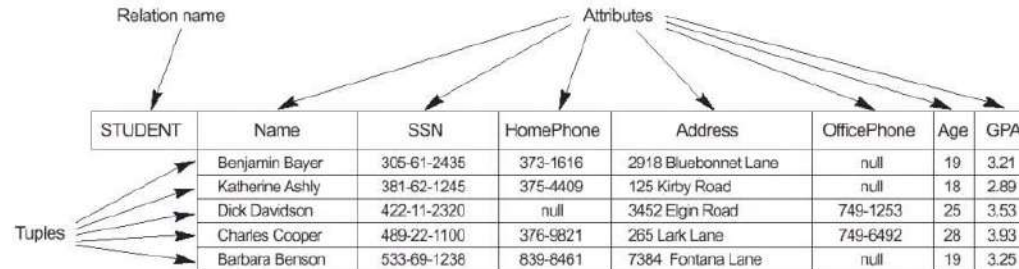
- **Database:**
 - A collection of related data.
- **Data:**
 - Known facts that can be recorded and have an implicit meaning.
- **Database Management System (DBMS):**
 - A software package/ system to facilitate the creation and maintenance of a computerized database.
- **Database System:**
 - The DBMS software together with the data itself. Sometimes, the applications are also included.

Terminology

- **Database** - contains one or more tables
- **Relation (or table)** - contains tuples and attributes
- **Tuple (or row)** - a set of fields which generally represent an “object” like a person or a music track
- **Attribute (also column or field)** - one of possibly many elements of data corresponding to the object represented by the row

Relations – cont...

- Student Table



STUDENT	Name	SSN	HomePhone	Address	OfficePhone	Age	GPA
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	null	19	3.21
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	null	18	2.89
	Dick Davidson	422-11-2320	null	3452 Elgin Road	749-1253	25	3.53
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	null	19	3.25

- Alternative terminologies for the relational model

Informal Terms

Table
Column
Row
Values in a column
Table Definition
Populated Table

Formal Terms

Relation
Attribute/Domain
Tuple
Domain
Relation Scheme
Relation Instance/Extension

Database System Architecture

Meaning

Database System Architecture describes the **structure of a DBMS** and explains **how data is stored, accessed, and managed** at different levels.

Objectives of Database Architecture

- Data abstraction
- Data independence
- Security and integrity
- Efficient data access
- Easy maintenance

Types of Database System Architecture

1. One-Tier Architecture

Description:

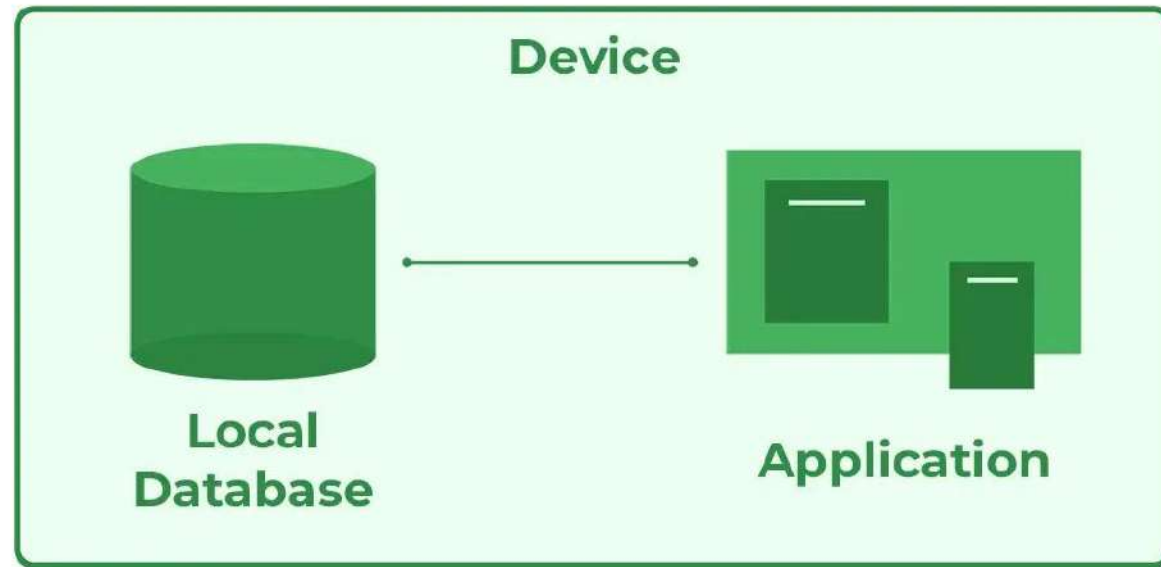
- Database, DBMS, and application are on the **same system**
- User directly interacts with the database

Features:

- Simple design
- No network required
- Used for learning and small applications

Example:

- MS Access
- SQLite



Two-Tier Architecture

Description:

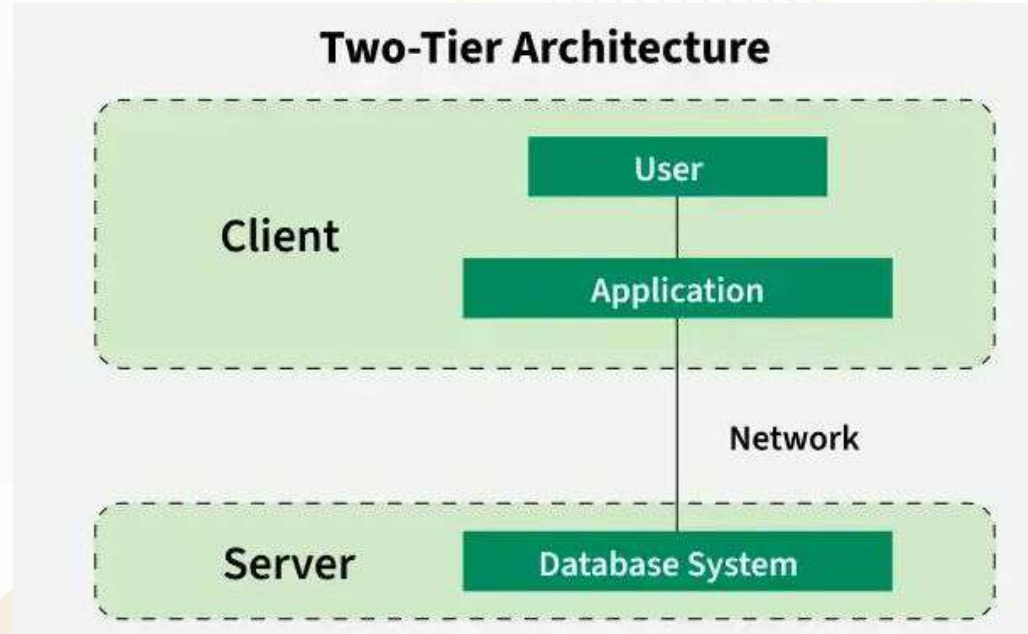
- Client directly communicates with the database server
- Application logic on client side
- Database on server side

Example:

- Desktop applications connected to MySQL

Limitations:

- Less secure
- Not suitable for large-scale applications



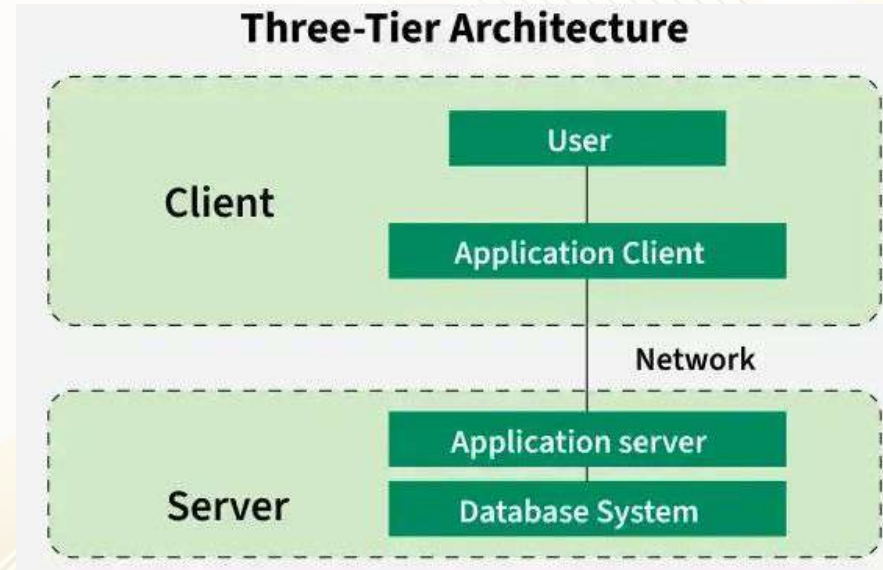
Three-Tier Architecture

Description:

- Most widely used architecture
- Application is divided into **three layers**

Layers:

1. **Presentation Layer** – User interface
2. **Application Layer** – Business logic
3. **Database Layer** – Database and DBMS



Advantages:

- High security
- Better scalability
- Easy maintenance

Example:

- Web applications
(Banking, E-commerce)

Database schema vs. Database instance

A **schema** acts as the blueprint for a database, describing its overall structure and how each element relates to another. In effect, it's an unchanging picture of how the database is formally organized.

A **database instance**, on the other hand, is a snapshot of the information contained within a database at a specific time. This means that, unlike a schema, the information contained within a database instance can change over time.

Three-Schema Architecture Purpose

Provides **data abstraction** and **data independence**

- **External Schema**
- **Conceptual Schema**
- **Internal Schema**

External Level (User Views)



Conceptual Level (Logical Schema)



Internal Level (Physical Storage)

External Level

- User view of the database
- Different users see different views
- Hides unnecessary data

Example:

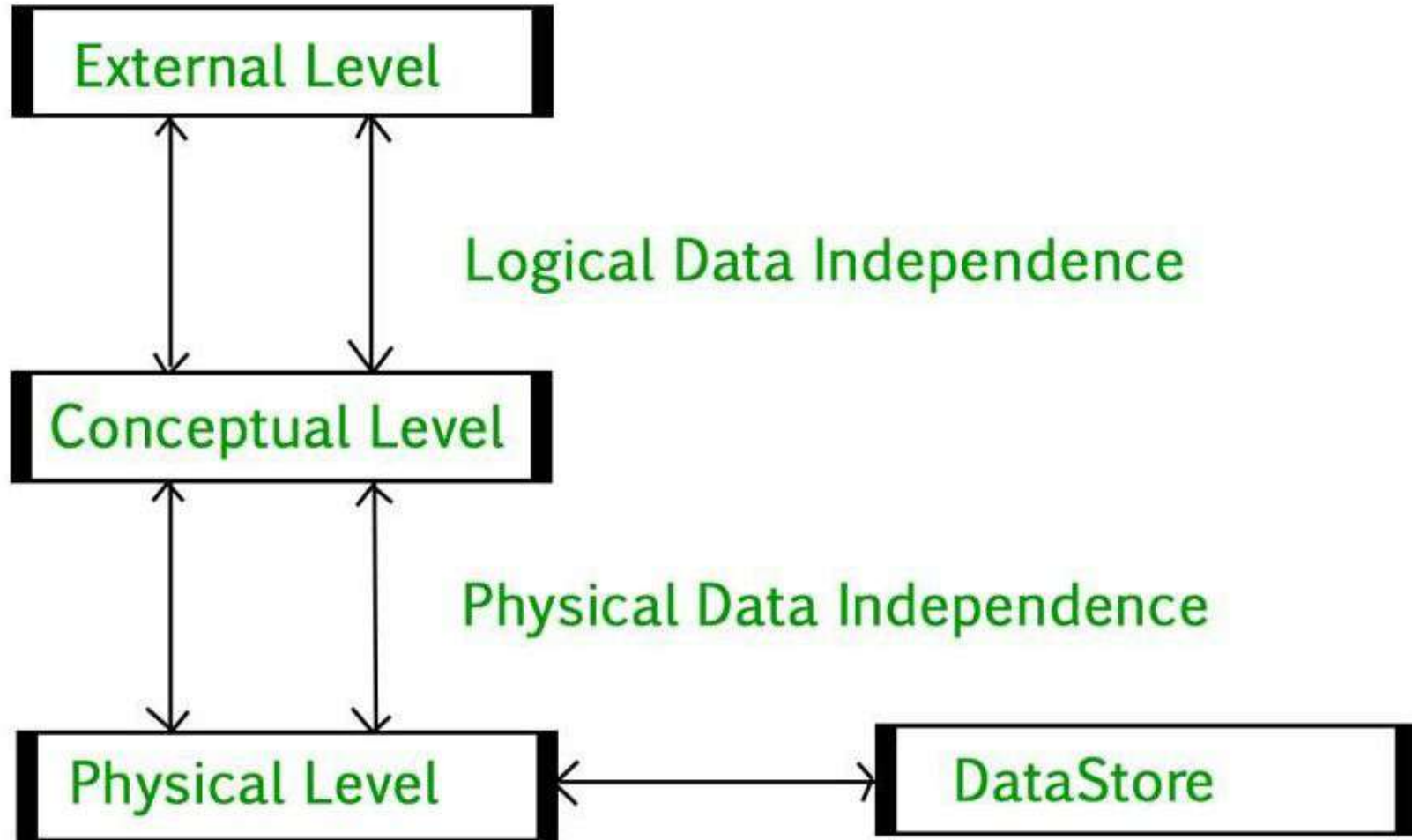
Student sees marks, Admin sees full records

Conceptual Level

- Logical structure of the entire database
- Defines tables, relationships, constraints
- Independent of physical storage

Internal Level

- Physical storage of data
- Files, indexes, memory allocation
- Concerned with performance



Data Independence

Definition:

Ability to change one level without affecting the other levels.

Types

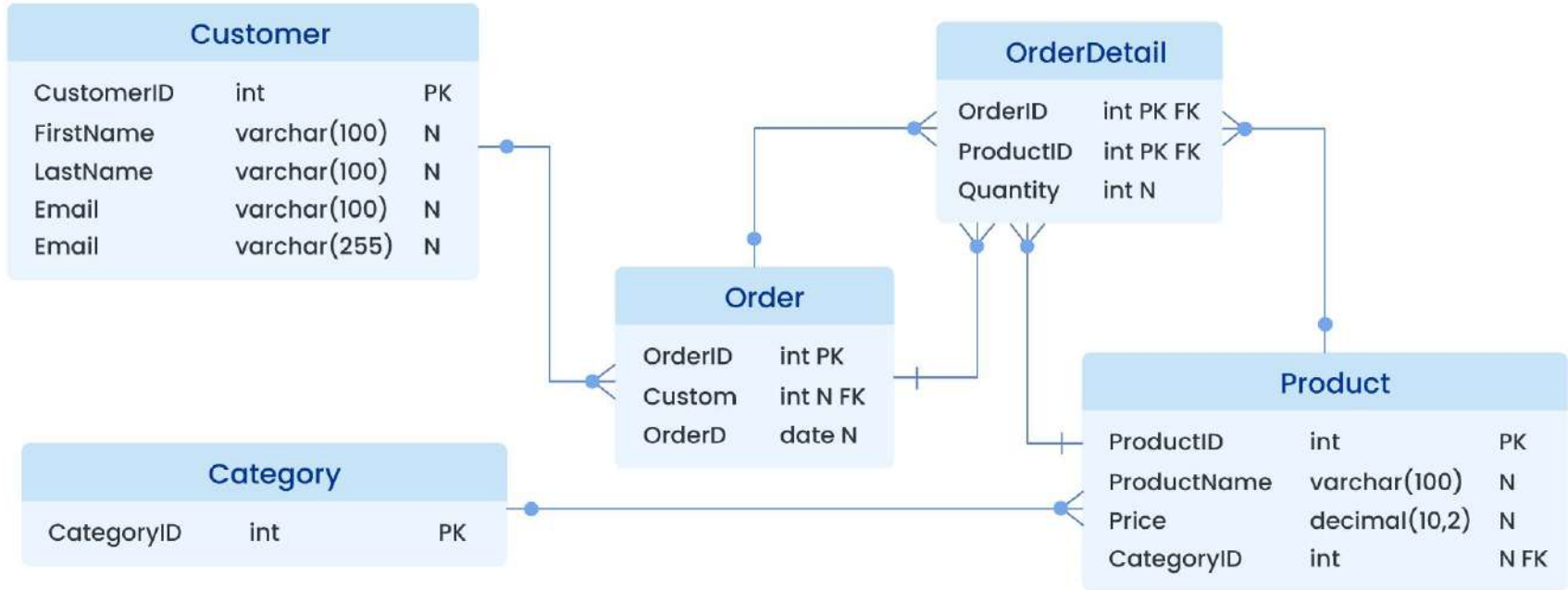
1. Physical Data Independence

- Change storage structure
- No impact on logical schema

2. Logical Data Independence

- Change logical schema
- No impact on user views

Other example of Relational Schema



In database terminology, what is an attribute?

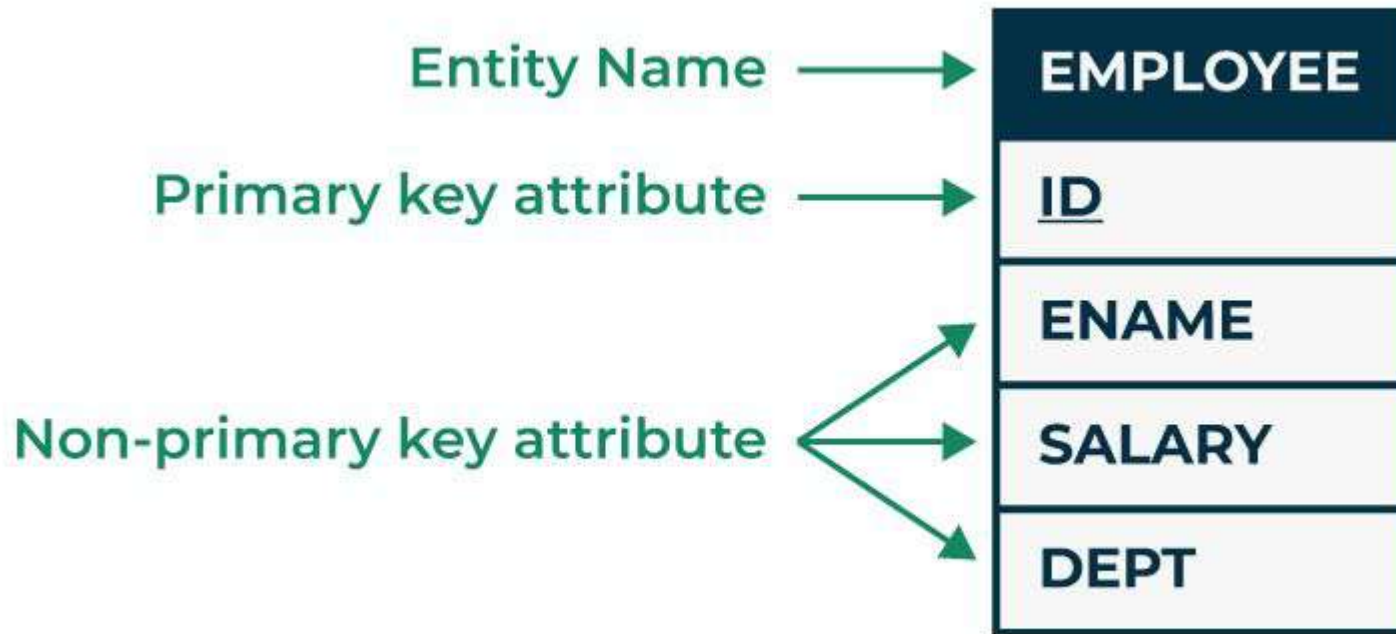
- A) A unique identifier for a record in a table
- B) A field or column in a table that holds data
- C) A table that stores multiple records
- D) A row in a table that contains data values

In database terminology, what is a relation?

- A) A unique identifier for a record in a table
- B) A row in a table that contains data values
- C) A table in a database that consists of rows and columns
- D) A field or column in a table that holds data

In database terminology, what is a tuple?

- A) A unique identifier for a record in a table
- B) A table that stores multiple records
- C) A row in a table that contains a set of values for each attribute
- D) A field or column in a table that holds data



Typical DBMS Functionality

- **Define** a particular database in terms of its **data types, structures, and constraints**
- **Construct or Load** the initial database contents on a secondary storage medium
- Manipulating the database:
 - **Retrieval:** Querying, generating reports
 - **Modification:** Insertions, deletions and updates to its content
 - **Accessing the database through Web applications**
- **Processing and Sharing** by a set of concurrent users and application programs – yet, keeping all data valid and consistent

#Example - Define a particular database in terms of its **data types, structures, and constraints**

```
CREATE TABLE Customers (  
    customer_id INTEGER PRIMARY KEY,  
    customer_name VARCHAR(100) NOT NULL,  
    date_of_birth DATE NOT NULL,  
    account_balance DECIMAL(10, 2) CHECK (account_balance >= 0)  
);
```

#Example - Construct or Load Initial Database Contents

A. Creating the Database

- Create the Database Schema

```
CREATE DATABASE CustomerDB;  
USE CustomerDB;
```

```
CREATE TABLE Customers (  
    customer_id INTEGER PRIMARY KEY,  
    customer_name VARCHAR(100) NOT NULL,  
    date_of_birth DATE NOT NULL,  
    account_balance DECIMAL(10, 2) CHECK (account_balance >= 0)  
);
```

- Load Initial Data

```
INSERT INTO Customers (customer_id, customer_name, date_of_birth, account_balance)  
VALUES
```

```
(1, 'John Doe', '1985-05-15', 1000.00),  
(2, 'Jane Smith', '1990-07-22', 1500.50),  
(3, 'Alice Johnson', '1988-11-03', 1200.75);
```

B. Using a Secondary Storage Medium

- Export the Database
- Backup to External Storage

C. Manipulating the Database

Example of Database

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

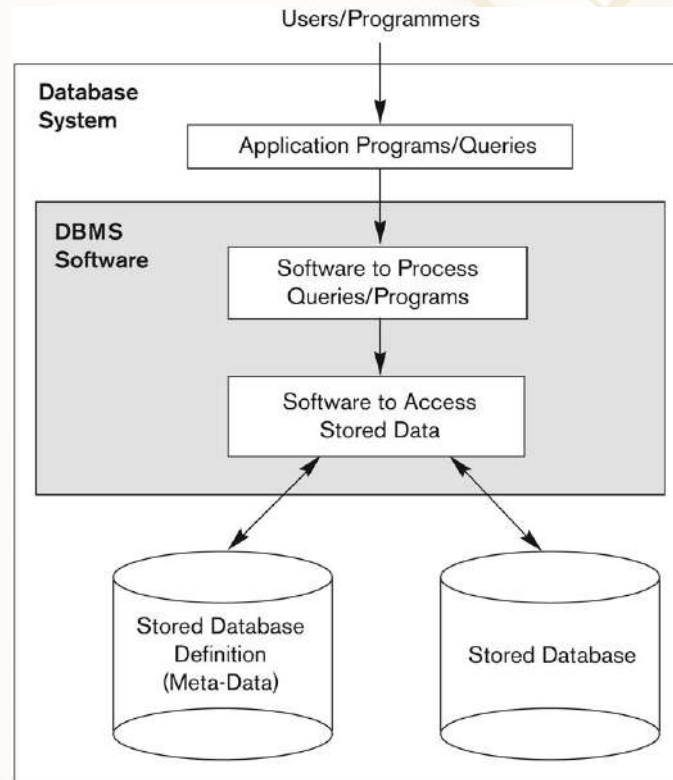
Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

s

Simplified Database Environment systems



Data Model / Database Schema and Database Instances

Database Schema: Logical Design of the Database

- Defines **how data in a database is structured**
- Defines how elements like foreign and primary keys, data types, and fields within a **database relate to each other**
- Accomplishes the above through the implementation of **coded rules**

Data Model / Database Schema and Database Instances

Database Instance: Snapshot of the data in the database at a given instance in time.

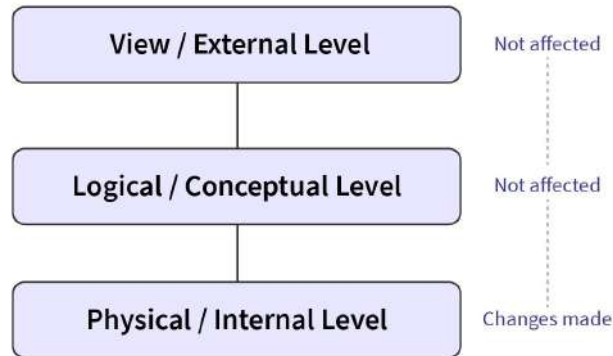
Data Independence in DBMS is of two types:

- 1. Physical Data Independence**
- 2. Logical Data Independence**

Data Independence

Physical Data Independence:

This is defined as the ability to modify the physical schema of the database without the modification causing any changes in the logical/conceptual or view/external level.



Why Physical Data Independence is important ?

- Physical data independence allows you to distinguish between conceptual and internal/physical levels. It allows you to **describe the database logically** without needing to identify physical structures.
- Physical data independence allows you to **modify physical storage structures or devices without affecting the conceptual model of the database**.

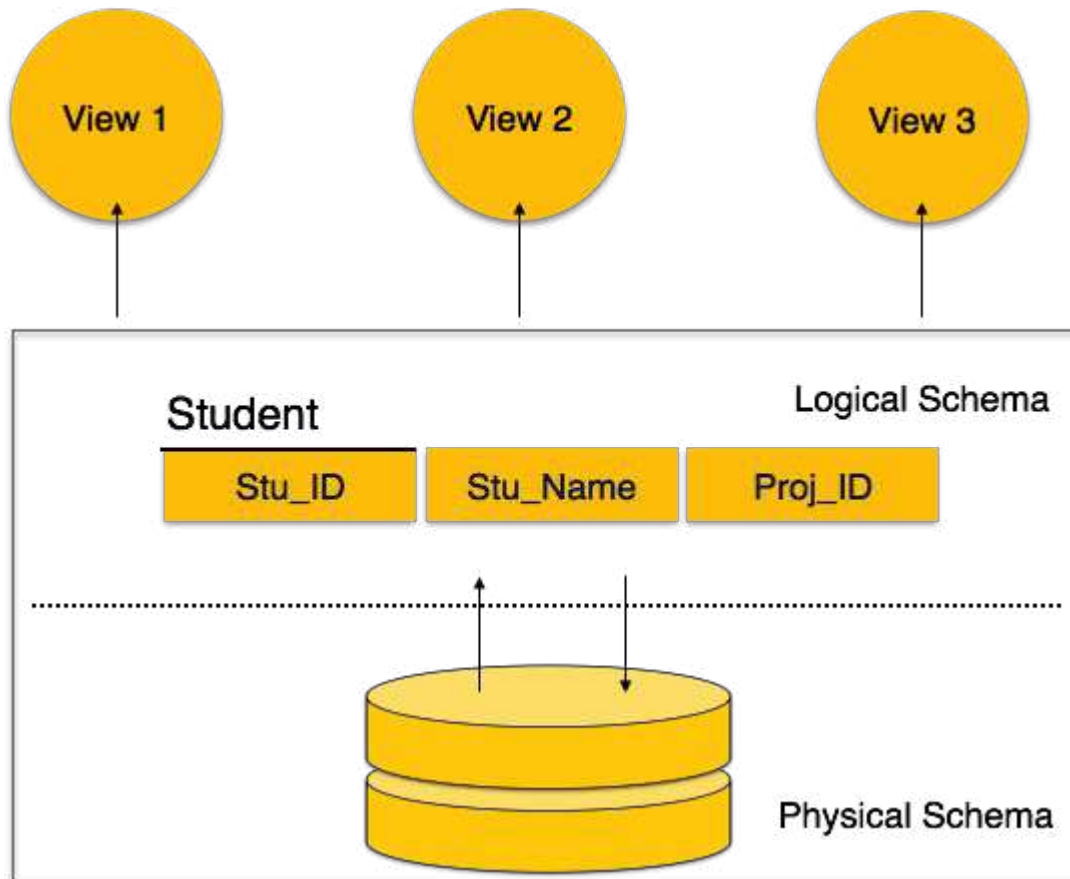
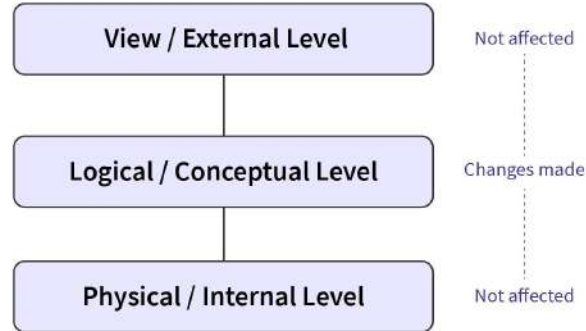


Figure: Three level architecture of DBMS

Data Independence

Logical Data Independence

Logical data independence is the **ability to modify logical schema without causing any unwanted modifications to the external schema** or the application programs to be rewritten.



Simple Example : Data Independence

Imagine you have a **library system** where you keep track of books and their details. The system is organized into a **catalog** where each book has information like the title, author, and publication year.

Logical Schema

Think of the **logical schema** as the structure of this catalog. It's how the information is organized internally. For example:

- Each book entry in the catalog has fields for title, author, and publication year.
- This catalog is stored in a database table named **Books**.

External Schema

The **external schema** is how the catalog is presented to users and how they interact with it. For example:

- Library staff might use a software program to add new books or check out books.
- Readers might use a website to search for books.

Logical Data Independence in Action

Let's say you want to improve the catalog by adding a new field for "Genre" to better categorize books. You make this change to the logical schema so that each book entry now also includes the genre.

Logical Data Independence means that you can make this change (add a "Genre" field) without affecting how library staff use their software or how readers search for books on the website. The software and website continue to work as they did before; they just see the additional "Genre" information when they query the catalog.

Why Logical Data Independence is important ?

- Logical data is database data, which means it **stores information** about **how data is managed within the database**. Logical data independence is a method that **makes sure that if we make modifications to the table format, the data should not be affected**.
- In other words, to distinguish the external level from the conceptual view, logical data independence is used. Any modifications to the conceptual representation of the data will not affect the user's view of the data.

Overview of DDL [Data Definition Language] and DML [Data Manipulation Language]

DDL [Data Definition Language]

Data Definition Language (DDL) commands allow me to **define and manage** a **schema** in SQL.

In a nutshell, a schema in SQL is a blueprint that defines how data is organized in a database and how the relationships among different tables within the database are managed.

DDL [Data Definition Language]

Data Definition Language (DDL) commands allow me to **define and manage a schema** in SQL.

In a nutshell, a schema in SQL is a blueprint that defines how data is **organized** in a database and how the **relationships among different tables** within the database are managed.

DDL Command	DESCRIPTION	SYNTAX
CREATE	Create a table and its columns together with their datatype.	CREATE TABLE
ALTER	Modify column names and add or delete a column.	ALTER TABLE
RENAME	Change the name of the table.	RENAME TABLE
COMMENT	Add an explanation to the SQL code for other team members to review.	-- /**/
TRUNCATE	Remove data from a table without deleting the table.	TRUNCATE TABLE
DROP	Delete the table with its data.	DROP TABLE

Data Manipulation Language

Data Manipulation Language, also known as **DML**, is a set of SQL commands that are used to **manipulate data within database tables or query views**.

Data analysts, scientists, engineers, and anyone using SQL rely on DML in order to **access, transform, and analyze data**.

Command	Description
SELECT	SELECT is an access command. It allows you to access data within a database.
INSERT	INSERT is a change command. It allows you to input rows of data within an existing table.
DELETE	DELETE is a change command. It allows you to remove rows of data within an existing table.
UPDATE	UPDATE is a change command. It allows you to change data within the rows of a table.

Data Control Language

Data Control Language (or DCL) consists of statements that **control security and concurrent access** to table data.

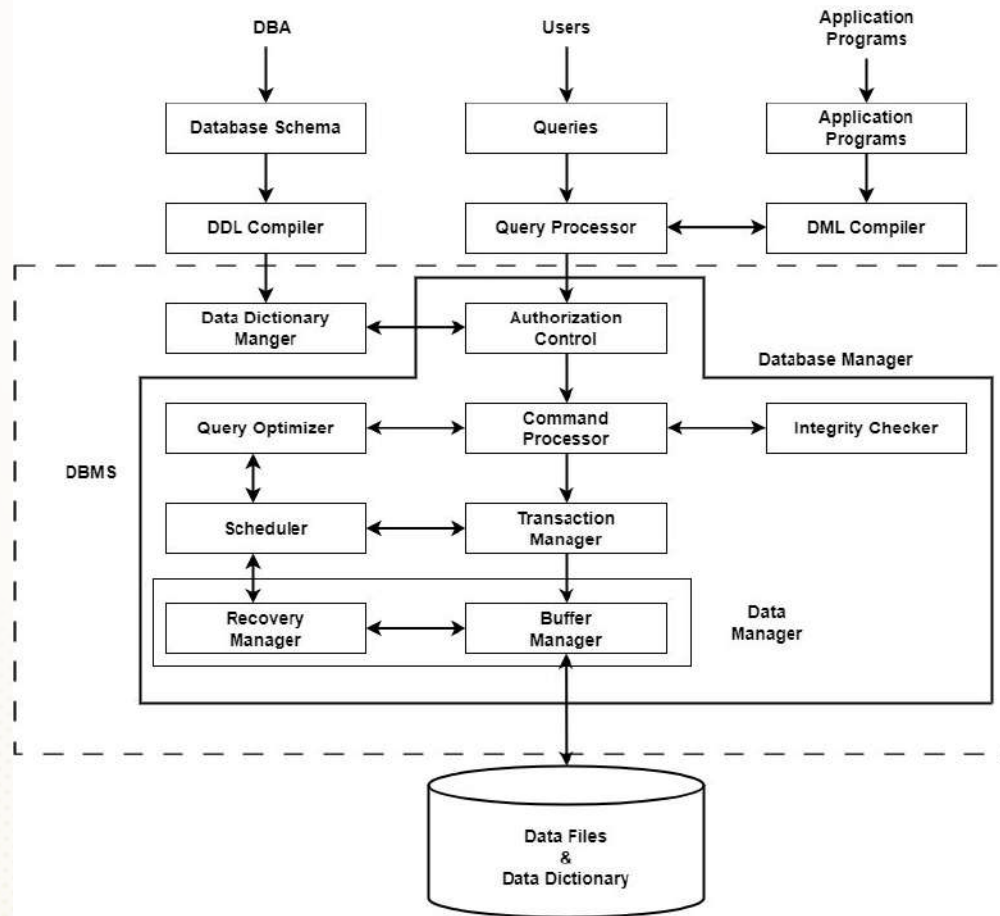
Database administrators (DBAs) and security teams use DCL to **control who can access and modify data** in a database and It ensures that sensitive data is protected by granting permissions only to authorized users.

Transaction Control Language

Transaction Control Language (TCL) consists of statements that **manage transactions in a database**, ensuring data integrity and consistency.

Database administrators (DBAs), developers, and data engineers use TCL commands to **control the execution of transactions**, allowing them to commit, roll back, or save changes in a controlled manner.

Database Structure



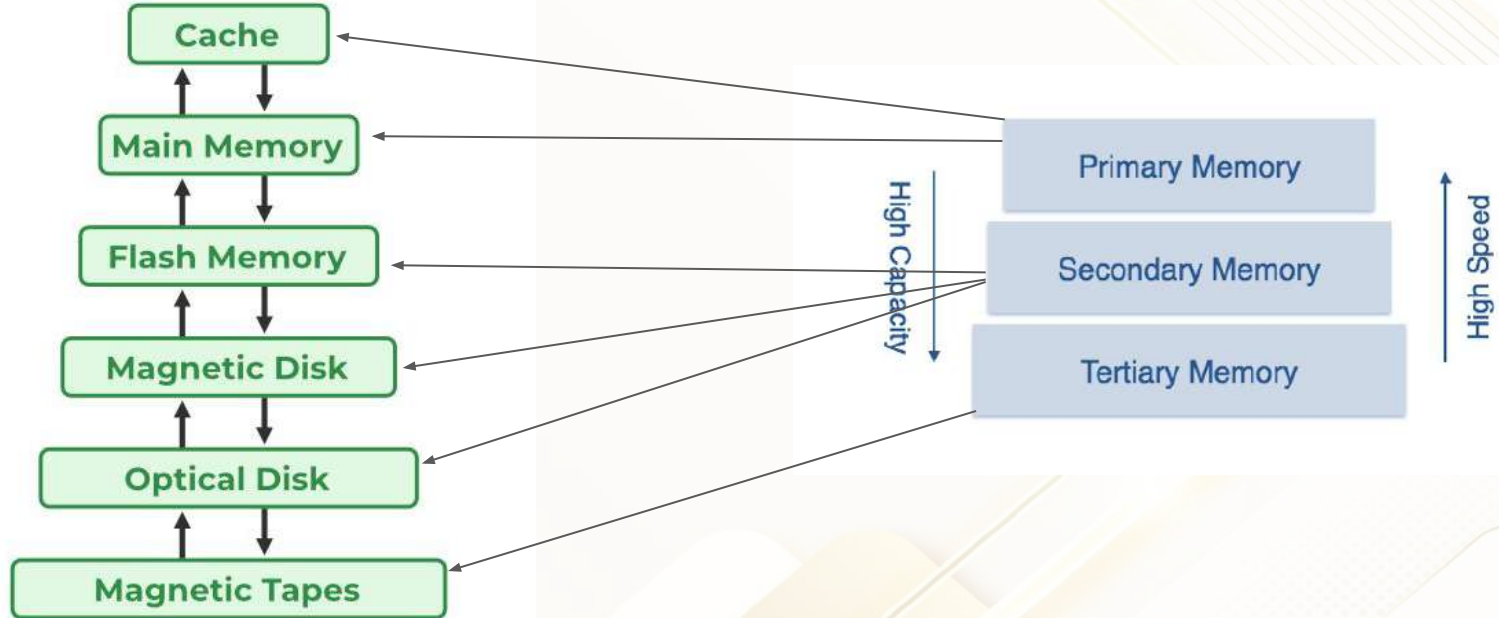
Overview of Storage Components

Data storage in a DBMS is broadly classified into **three categories based on the nature of data persistence and speed of access**:

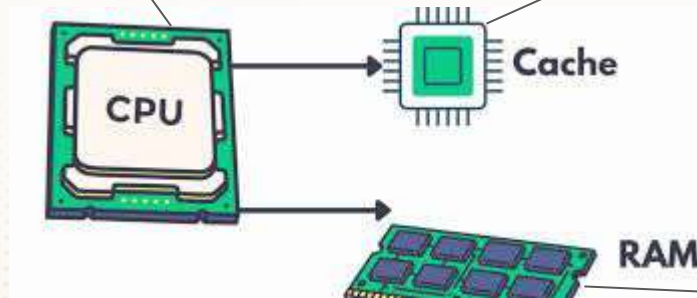
1. Primary Storage
2. Secondary Storage
3. Tertiary Storage

Each type of storage serves a specific function in maintaining the balance between fast data access and reliable long-term storage. These storage types vary significantly in terms of performance, cost, and volatility.

Overview of Storage Components



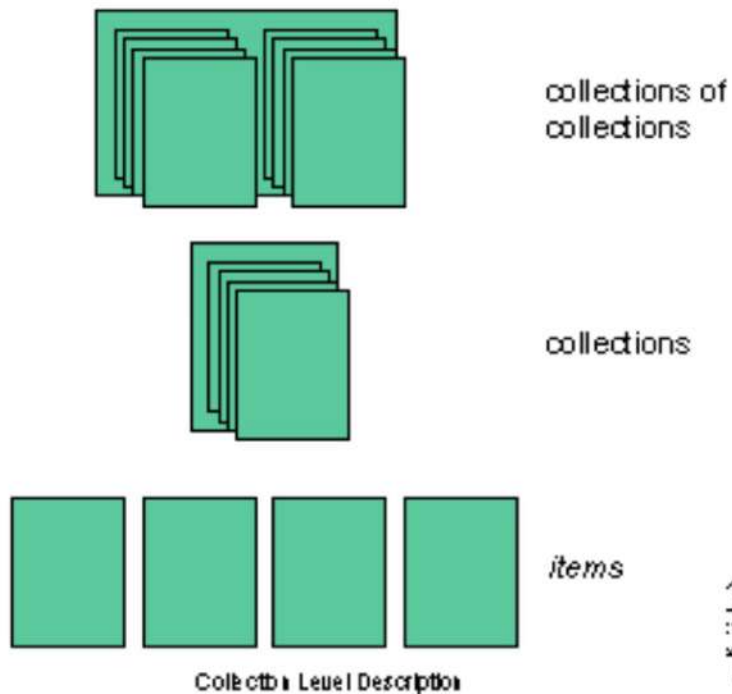




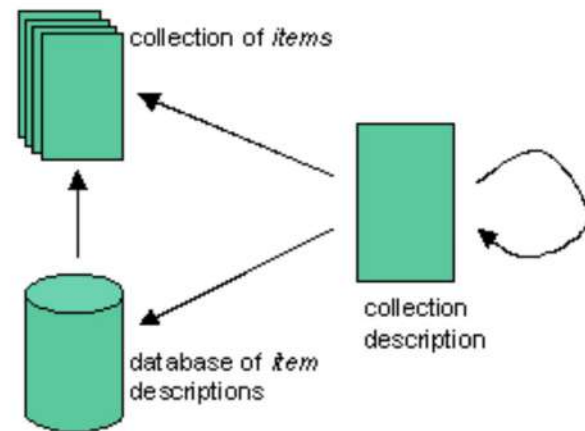
Overview of Metadata

1. Data about data.
2. Data that describes how user data are stored in terms of table name, column name, data type, length, primary key and e.t.c.
3. Metadata are typically stored in system tables and typically accessible by the DBMS or by the system administrator.

granularity



data vs. metadata



Collection Level Description



Overview of recovery mechanism

Why Backup data ??

Backing up data is vital for businesses

- Lost information can cause a major crisis or lead to business failure

Common Problems

1. **System outage:** Power and hardware failure
2. **Transaction failure:** Users may inadvertently corrupt the database
3. **Media Failure:** Disk drive becomes unusable
4. **Disaster:** Database facility damaged by fire, flooding, or other catastrophe

Overview of recovery mechanism

Database Recovery

1. **Mechanism for restoring a database quickly and accurately after loss or damage**
2. **RESPONSIBILITY OF ?????** (highlighted in blue)
3. **Recovery facilities:**
 - a. Backup Facilities
 - b. Journalizing Facilities
 - c. Checkpoint Facility
 - d. Recovery Manager

It seems to be discussing database recovery mechanisms and asking whose responsibility it is to ensure recovery. Likely, the missing answer is **"Database Administrator (DBA)"** or **"Recovery Manager"**.

MCQ [Recall and Revise]

Which of the following is NOT an advantage of a database system over a file system?

- A) Data redundancy and inconsistency reduction
- B) Data sharing and security
- C) **Faster direct file access**
- D) Better data integrity and consistency

In a database system, what does the term "Schema" refer to?

- A) The actual data stored in the database
- B) **The logical structure that defines how data is organized**
- C) A temporary data storage for transactions
- D) A collection of primary keys in a table

Which of the following is an example of a Data Manipulation Language (DML) command?

- A) CREATE
- B) ALTER
- C) **INSERT**
- D) GRANT

Which of the following is NOT a storage component of a database system?

- A) Index files
- B) Metadata tables
- C) **CPU cache**
- D) Data files

What is the primary purpose of database backups and recovery mechanisms?

- A) To delete unnecessary records
- B) To secure data from unauthorized users
- C) **To restore data in case of system failures or corruption**
- D) To improve query execution speed

Which of the following best describes the purpose of Data Independence?

- A) **It allows changes in data storage without affecting application programs**
- B) It ensures only one user can access the database at a time
- C) It restricts unauthorized access to the database
- D) It improves network communication speed

What is the primary purpose of database backups and recovery mechanisms?

- A) Writing application programs
- B) **Managing database storage and security**
- C) Executing queries only
- D) Designing network protocols

Which of the following is NOT a function of Metadata in a database?

- A) **Storing actual data records**
- B) Describing data types and constraints
- C) Defining relationships between tables
- D) Helping the DBMS understand data structure

Which type of database language is used to control access to a database?

- A) DDL (Data Definition Language)
- B) DML (Data Manipulation Language)
- C) **DCL (Data Control Language)**
- D) TCL (Transaction Control Language)

Which of the following statements about a Primary Key is true?

- A) A primary key can have duplicate values.
- B) A table can have multiple primary keys.
- C) **A primary key uniquely identifies each record in a table.**
- D) A primary key must always be a numeric column.

Practice Questions

Link : [Click here to open the document](#)

End of Unit 01 : Feel free to ask questions !!